

Team Polar Autonomous Sub-team Application Challenge

Introduction

The autonomous sub-team in Team Polar is the team who is responsible for the software which adds autonomous driving intelligence to our rovers. Autonomous driving is achieved by creating software functionalities which connect with all respective hardware interfaces like sensors, motors, on-board computers, the battery, and more.

In our daily developments, we utilise tools like ROS2, SLAM, algorithms like A* or Field D*, Kalman filtering, AI powered object-detection, PID control, fuzzy logic, model-predictive control, and more. Through a combination of it all, we aim to solve the 4 pipeline steps of autonomous driving — PPPA.

Perception -> Processing -> Planning -> Actuation

Some pipeline steps are more mathematical, some more computer-science-y, some more to do with mechanical control, and some more to do with electrical signals. Hence, no matter your major, if you're up to develop autonomous driving capabilities much in the same way as organisations like NASA, Tesla, or Waymo, we invite you to complete the assignment below.

Challenge Preface

The challenge is designed to evaluate your problem-solving approach, coding skills, and ability to learn new concepts. No prior robotics experience is required. The estimated time required is ~3h. There are also no restrictions on the use of AI, Google, or mom and dad's advice.

“Don't judge a fish by its ability to climb a tree” — we recognise that this assignment may not properly capture your skills. For example, maybe you're not too familiar with Python syntax, but you're really good at C or MATLAB. Thus, to take a more holistic approach at assessing you as a candidate, we would like you to also submit a short a reflection alongside your code. The reflection may include some of the following:

- Description of how you tackled the problem.
- Explanation of your solution.
- How would you assess the difficulty of this problem?
- If any, what resources did you use to help yourself?
- Any other information that you think we should take into consideration.

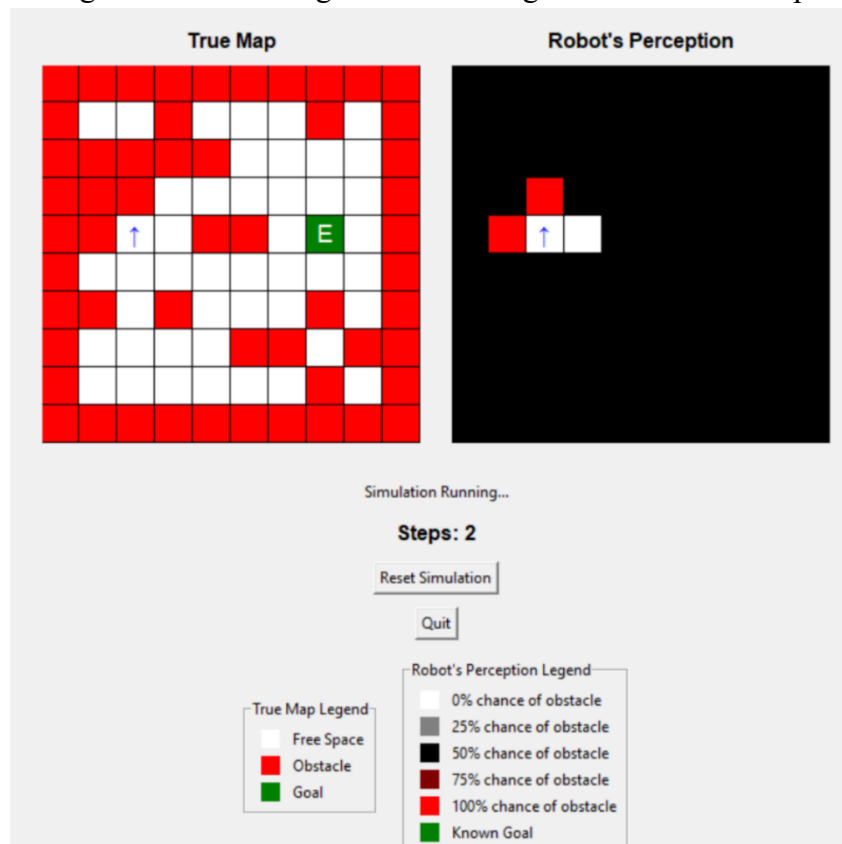
In the end, we also hope you enjoy this challenge and perhaps learn something new as you solve it.

Submission Details:

- Submit your code as a .py file named “name_code.py”.
- Submit the reflection as “name_reflection.pdf”
- Please zip your files and name the folder according to the instructions in the email.

Challenge Description

The challenge asks you to write an automated Python script which helps a robot navigate a digital maze. The goal of the challenge is to program a solution which helps the robot get from its starting tile to the end goal tile in as few steps as possible.



Consider the following details about the challenge:

- The robot is denoted by a blue arrow. The robot has an orientation which is either NORTH, SOUTH, EAST, or WEST.
- The robot lives in a grid-like world, which is anywhere between 9x9 and 14x14
- The world consists of obstacles (red), free space (white), and an end goal (green). The robot may move freely between adjacent white tiles, but crashes if it attempts to move onto a red tile.
- On any given step, the robot can complete one of the following actions:
 - Re-orient itself (either return “N”, “S”, “E”, or “W”)
 - Move in the direction it is facing (return “M”)
 - Capitalization matters.
- Unfortunately, the robot’s sensors are malfunctioning. It can only see the contents of the following tiles:
 - The tile the robot itself is on
 - The two tiles adjacent to the robot
 - The three tiles in front of the robot

The sensors can only detect obstacles with a certain confidence. The robot always knows what kind of tile it is on itself, but for the other 5 tiles it gives a confidence score for the existence of an obstacle on that tile. The confidence of an obstacle occupying a tile can be either 0, 0.25, 0.5, 0.75, or 1. Every time the robot completes an action, these confidence scores are re-generated. If at a certain point the robot was

100% sure that there was obstacle on the tile in front of it, then that obstacle is definitely there, although the sensors may give a different confidence later on.

- In a GUI you can view two maps: the true map, and the robot's perception map. However, in your code implementation, you only have access to the perception map. The true map may simply help with debugging.
- When the world is generated, there is no guarantee that a path exists between the starting location and the end location. In such a case, simply re-launch the simulation.
- Your solution should be entirely implemented in the appropriate part of the "name_code.py" file, but you may also view the contents of the "robot_simulation.py" file.
- Dependencies: Python 3, NumPy.
- The simulation code hasn't been extensively tested. Bugs may exist. Some of them have been identified but not yet fixed.
 - We've been informed that the reset button works improperly depending on how the programmer writes their code to interact with the simulation. In such cases, simply relaunch the simulation rather than attempting to use the button.
 - The 'game_state.current_position' function returns coordinates as (x,y), whereas the 'game_state.perception_matrix' returns them as (y,x).
 - The game_state.current_orientation function does not return 'NORTH' as stated, but instead 'Orientation.NORTH'. Naturally, this applies to all cardinal directions.

The details above are simply a breakdown of how the simulator works. Any approach to the problem is accepted. In addition to looking at the reflection, we will test your code on 10 standardised pre-made maps. Prioritise ability to solve maps over speed.